

Turfight Planner

複数募集回データモデル・永続化設計

Document ID: ARCH-TURFIGHT-073

Status: Approved

Version: 1.1.0

Source of Truth: Turfight_Planner_Multiple_Recruitment_Rounds_Architecture.md

Markdown SHA-256:

88de9e8abee27e273d56dc3942da6e2edfb5647231a90eb7e936aac7f7fetc09

Generated: 2026-07-12T20:10:46+09:00

Issue #73 approved architecture artifact

Turfight Planner 複数募集回データモデル・永続化設計

1. エグゼクティブサマリー

Issue #73に対し、複数募集回を安全に扱うデータモデルとブラウザ永続化戦略を設計した。調査対象は E:\Document\Develop\turfight-planner\index.html のコミット b739ee2 であり、作業ツリーに未コミット変更がない状態を基準とした。

Product Owner決定に基づく推奨は、案A: 募集回集約モデル である。階層を Application -> RecruitmentRounds -> RecruitmentRound -> Plans -> HorseSelections とし、schemaVersion を4へ上げる。募集回はPlanの親エンティティとし、今回上限、金券、支払方法を募集回設定として保持する。表示フィルターはPlanへ、馬の選択入力はPlan配下へ置く。

Rootは activeRecruitmentRoundId を、各Recruitment Roundは activePlanId を保持する。保存時は現在募集回と現在Planだけを既存Rootへマージし、読込時はRootから募集回、募集回からPlanの順に選択状態を解決する。現行schemaVersion 3はPlanを親としているため、移行時にPlan×募集回の状態を募集回ごとのPlanへ展開する。

2. 目的と範囲

2.1 目的

複数の募集回について、各募集回に所属するPlanの入力状態を独立して保存・復元し、既存利用者の保存データを失わずに承認済み階層へ移行できるアーキテクチャを定める。

2.2 対象

- ・ 現行のブラウザ内永続化モデル
- ・ getPlanData()、applyPlan()、募集回設定の責任
- ・ schemaVersion 3以前からの移行
- ・ 3つの設計代替案と推奨案
- ・ 推奨JSON形式
- ・ 実装用の後続Issue分割

2.3 対象外

- ・ コード変更、テスト追加、UI実装
- ・ サーバー同期、ユーザー認証、複数端末同期
- ・ 募集馬カタログの取得方式
- ・ コミット、ブランチ作成、Pull Request

3. 記述区分

区分 意味
Fact 現行コード、Git状態、Product Owner決定から直接確認した事実
Assumption 承認済み判断を具体化するために置いた技術前提
Proposal 本調査で推奨する将来設計。現行実装の事実ではない

4. 現行実装の調査結果

4.1 調査基準

Fact

- 対象リポジトリ: E:¥Document¥Develop¥turflight-planner
- 基準コミット: b739ee2 (origin/main と同一)
- 調査時ブランチ: codex/issue-36-recruitment-header
- 調査時の作業ツリー: clean
- アプリケーションは単一の index.html で構成される。

4.2 永続化モデル

Fact

- 保存先は localStorage の単一キー hitokuchiPlanner.card.v5.fixed である (index.html:554)。
- 保存ルートは { schemaVersion: 3, activePlanId, plans } である (index.html:646-663)。
- JSON読込失敗時は例外を通知せず空オブジェクトを返す (index.html:665-671)。
- 保存失敗時も例外を通知しない (index.html:701-704)。
- Plan追加、名前変更、削除も同じキーヘルート全体を書き戻す (index.html:803-853)。

4.3 現行Planモデル

Fact

getPlanData() が生成するPlanは次の責任を持つ (index.html:624-643)。

フィールド 現在の配置 内容
id, name Plan直下 Plan識別・表示名
eventBudget Plan直下 今回上限
bulkRate Plan直下 支払方法
voucherAmount Plan直下 金券
viewFilter Plan直下 表示条件
activeRecruitmentRoundId Plan直下 現在募集回ID。ただし現状は先頭回のみ
recruitmentRounds Plan直下のMap 募集回別の馬入力

recruitmentRounds[roundId] は { id, name, horses } であり、horses[no] は { units, memo } を保持する (index.html:596-605, 638-642)。馬の名称・価格・総口数などのカタログ情報はHTMLにあり、保存JSONには複製されない。

4.4 募集回の検出と選択

Fact

- DOM上の [data-recruitment-round-id] を募集回ルートとして列挙する (index.html:558-580)。
- 現在のHTMLには data-recruitment-round-id="2026-current" のルートが1件だけある (index.html:118)。
- getCurrentRecruitmentRound() は列挙結果の先頭要素を無条件で返す (index.html:588-590)。
- data-recruitment-round-name がない場合、表示名は 現行募集回 となる (index.html:572)。

DOMへ複数募集回を追加しただけでは2件目以降を選択できず、保存・読込・集計・フィルターも先頭回だけを対象にする。

4.5 getPlanData() の挙動とデータ消失リスク

Fact

getPlanData() は先頭募集回の馬入力だけを読み、空の recruitmentRounds からPlanを新規構築する。
getSaveData() は既存PlanをこのPlanで全体置換する (index.html:624-657)。複数募集回が保存済みでも、

現在回以外のエントリを失うデータ損失経路である。

4.6 applyPlan() の挙動

Fact

- Plan直下の4設定を画面へ適用する (index.html:757-761)。
- 先頭募集回のIDを使って保存馬を取得し、その回のカードだけへ適用する (index.html:763-772)。
- activeRecruitmentRoundId は参照しない。
- 対象回にデータがなければ、旧形式の plan.horses へフォールバックする (index.html:608-613)。

4.7 募集回設定の現状

Fact

eventBudget、voucherAmount、bulkRate、viewFilter のUIは募集回ルートの外側にあり (index.html:97-102, 118)、データはPlan直下に保存される。集計と表示フィルターは先頭募集回のカードだけを対象にする (index.html:1043-1055)。

5. 承認済み判断と設計要件

5.1 Product Owner Decisions

Fact - Decision D1

Recruitment Roundは親エンティティであり、Planは必ず1つのRecruitment Roundに所属する。

Fact - Decision D2

eventBudget、voucherAmount、bulkRate はRecruitment Round設定である。

Fact - Decision D3

viewFilter はPlan設定である。

Fact - Decision D4

Rootが activeRecruitmentRoundId を持ち、各Recruitment Roundが activePlanId を持つ。

Fact - Decision D5

馬カタログデータとユーザー選択を分離する。カタログは永続状態へ複製しない。

5.2 技術前提

Assumption A1

roundId はRoot内で一意かつリリースをまたいで安定する。planId は所属募集回内で一意である。馬IDは募集回内で一意かつ安定する。

Assumption A2

schemaVersion 3の同一Planが複数募集回を保持する場合、移行後は各募集回に同名・同IDのPlanを1件ずつ作る。Plan IDの名前空間は募集回に閉じるため、募集回間の同一IDを許容する。

5.3 機能要件

1. 複数のRecruitment Roundを列挙し、Rootで1件を選択できる。
2. 各Recruitment Round内で複数Planを管理し、1件を選択できる。
3. Recruitment Roundごとに予算、金券、支払方法を保存する。
4. Planごとに表示フィルターと馬選択を保存する。

5. 再読み時に最後のRecruitment Roundと、その回で最後に選択したPlanを復元する。
6. DOMに存在しない過去募集回の保存データを通常保存で削除しない。
7. schemaVersion 3以前のデータを決定的かつ幂等に移行する。

5.4 品質要件

- 保存操作で他Recruitment Round・他Planを欠落させない。
- カタログ情報とユーザー入力を分離する。
- 破損JSON、未知フィールド、将来schemaVersionを検出する。
- 移行中に元データを上書きする前に検証する。
- 単一HTML・ブラウザーローカル利用へ過剰な基盤を導入しない。

6. 設計代替案

6.1 案A: 募集回集約モデル

```
Application
├── RecruitmentRounds{roundId}
│   └── RecruitmentRound
│       ├── settings, activePlanId
│       ├── Plans[]
│       └── Plan
│           ├── viewFilter
│           └── HorseSelections{horseld}
```

Proposal

承認済み業務階層をそのまま永続モデルにする。Rootは募集回選択、募集回は設定とPlan選択、Planは表示設定とユーザー選択を所有する。

長所:

- Product Owner決定と責任境界が一致する
- 募集回設定をPlan間で重複保存しない
- Rootから募集回、募集回からPlanというUI選択順と一致する
- ある募集回の保存で他回を保持しやすい

短所:

- 現行Plan親モデルから階層を反転する移行が必要
- 旧Planが複数回を持つ場合、募集回ごとのPlanへ展開する必要がある

6.2 案B: ルート正規化モデル

```
Application
├── RecruitmentRounds{roundId}
├── Plans{planKey}
└── HorseSelections{planKey}
```

Proposal

募集回、Plan、馬選択をRootの別コレクションへ置き、外部キーで関連付ける。

長所:

- エンティティ単位の横断検索と将来の外部DB移行に適する
- データ重複が少ない

短所:

- ・親子関係と参照整合性をアプリ側で管理する必要がある
- ・Plan削除時の孤児選択など、単一HTML用途には複雑
- ・JSONの診断性と移行単純性が案Aより低い

6.3 案C: 募集回別localStorageキー

```
app.index -> activeRecruitmentRoundId, round index  
app.round.{roundId} -> settings, activePlanId, plans
```

Proposal

Rootインデックスと各募集回集約を別キーへ保存する。

長所:

- ・募集回単位で読み書きできる
- ・1件の破損が他募集回へ波及しにくい

短所:

- ・localStorage にはトランザクションがなく、Rootと募集回が不整合になり得る
- ・移行、バックアップ、全消去が複数キー操作になる
- ・現状のデータ量では分割効果が小さい

6.4 比較

評価は5点を最良とする。

評価軸 重み 案A 募集回集約 案B 正規化 案C 複数キー
Product Owner決定との一致 30 5 4 5
移行の安全性・説明可能性 20 4 2 2
更新時の整合性 20 5 3 2
将来拡張性 15 4 5 4
実装・検証コスト 15 4 2 2
加重評価 (5点満点) 100 4.45 3.30 3.10

7. 推奨アーキテクチャ

7.1 決定

Proposal P1

案AをschemaVersion 4として採用する。永続階層は Application -> RecruitmentRounds -> RecruitmentRound -> Plans -> HorseSelections とする。localStorage キーはデータを見失わないため hitokuchiPlanner.card.v5.fixed を維持する。

7.2 責任境界

層 責任 保存対象
Application スキーマ版、最後に選択した募集回 schemaVersion, activeRecruitmentRoundId, recruitmentRounds
RecruitmentRound 募集回識別、回共通設定、最後に選択したPlan id, settings, activePlanId, plans
Plan 募集回内シナリオ、Plan固有表示設定 id, name, viewFilter, horseSelections
HorseSelection 募集馬ごとのユーザー入力 units, memo
HTML catalog 募集回名、馬名、金額、総口数など localStorageへ保存しない

7.3 推奨JSON例

```
{
  "schemaVersion": 4,
  "activeRecruitmentRoundId": "2026-second",
  "recruitmentRounds": {
    "2026-current": {
      "id": "2026-current",
      "settings": {
        "eventBudget": "300000",
        "voucherAmount": "20000",
        "bulkRate": "0.9"
      },
      "activePlanId": "default",
      "plans": [
        {
          "id": "default",
          "name": "本命プラン",
          "viewFilter": "selected",
          "horseSelections": {
            "15": { "units": "1", "memo": "優先候補" },
            "16": { "units": "0", "memo": "様子見" }
          }
        }
      ]
    },
    "2026-second": {
      "id": "2026-second",
      "settings": {
        "eventBudget": "200000",
        "voucherAmount": "0",
        "bulkRate": "0.95"
      },
      "activePlanId": "plan-a",
      "plans": [
        {
          "id": "plan-a",
          "name": "第二回候補",
          "viewFilter": "all",
          "horseSelections": {
            "1": { "units": "2", "memo": "最優先" }
          }
        }
      ]
    }
  }
}
```

値の型は現行DOM入力との互換性を優先し、Issue #73では文字列を維持する。数値型への正規化は別Issueとする。

7.4 保存戦略

Proposal P2

getPlanData() の「Plan全体を新規構築する」責任を廃止し、募集回設定のcaptureとPlan状態のcaptureを分離する。

```
UI input
-> captureRecruitmentRoundSettings()
-> captureActivePlanState()
```

```
-> read + migrate + validate Root
-> merge current RecruitmentRound settings
-> upsert current Plan in current RecruitmentRound
-> preserve inactive RecruitmentRounds and Plans
-> serialize Root once
```

不変条件:

- 現在募集回以外を変更しない。
- 現在Plan以外を変更しない。
- 募集回設定はPlanへ複製しない。
- viewFilter はPlan以外へ保存しない。
- DOMに存在しない募集回・馬選択を暗黙削除しない。
- captureまたは検証に失敗した場合は書き込まない。

7.5 読込・適用戦略

Proposal P3

読込は次の順序で行う。

1. Rootの activeRecruitmentRoundId を解決する。利用不能ならカタログ先頭回へフォールバックする。
2. 選択したRecruitment Roundの設定をUIへ適用する。
3. その回の activePlanId を解決する。利用不能ならその回の先頭Planへフォールバックする。
4. 選択Planの viewFilter と horseSelections を対象回DOMへ適用する。
5. 集計とフィルターを再実行する。

applyPlan() は募集回設定を扱わず、所属募集回が確定した後にPlan固有状態だけを適用する。募集回設定は applyRecruitmentRound() が担当する。

7.6 切替戦略

Proposal P4

募集回切替:

1. 旧募集回の設定と旧Plan状態を保存する。
2. Rootの activeRecruitmentRoundId を新IDへ変更する。
3. 新募集回の設定を適用する。
4. 新募集回の activePlanId が示すPlanを適用する。
5. 再計算・フィルター適用を行う。

Plan切替:

1. 旧Planの viewFilter と horseSelections を保存する。
2. 現在Recruitment Roundの activePlanId を新IDへ変更する。
3. 新Plan固有状態を適用する。
4. 再計算・フィルター適用を行う。

選択IDを先に変更してから旧画面を保存すると別エンティティへ誤保存するため禁止する。

8. 移行戦略

8.1 方針

Proposal P5

読み直後に純粋関数 `migrateToV4(rawData, catalogContext)` でメモリ上の形式を正規化する。移行は冪等、非破壊、決定的、失敗安全とする。検証成功後の初回ユーザー起点保存時にv4を書き込む。

8.2 schemaVersion 3から4

1. 全旧Planの `recruitmentRounds`、`activeRecruitmentRoundId`、カタログ先頭回からround IDの和集合を作る。
2. 各round IDについてRecruitment Round集約を作る。
3. 旧Planがそのroundの馬入力を持つ場合、その募集回内へ同じ id と name のPlanを作る。
4. 馬入力を `horses` から `horseSelections` へ名前変更して保持する。
5. 旧Planの `viewFilter` を生成した各Planへ保持する。
6. `eventBudget`、`voucherAmount`、`bulkRate` は募集回設定へ移す。
7. Rootの旧 `activePlanId` が各募集回内に存在すれば、その回の `activePlanId` とする。存在しなければ先頭Planとする。
8. Rootの `activeRecruitmentRoundId` は旧active Planの `activeRecruitmentRoundId`、カタログ先頭回の順で解決する。
9. Rootの `schemaVersion` を4にする。

旧形式では回共通設定がPlanごとに異なり得るため、次の優先順位でRecruitment Round設定を決定する。

1. 旧Rootの `activePlanId` と一致し、その募集回データを持つPlan
2. その募集回データを持つ旧Plans配列の先頭Plan
3. 旧active Plan
4. 既定値

競合値は自動統合せず、選択元と競合件数を移行診断として記録する。元Planの馬入力と表示フィルターは全て保持する。

8.3 schemaVersion 2以前・版なし形式

旧ルートデータを1つのRecruitment Roundへ包み、その中に default Planを作る。round IDは旧 `activeRecruitmentRoundId`、カタログ先頭回、`default-round` の順で解決する。旧 `eventBudget`、`voucherAmount`、`bulkRate` は募集回設定へ、`viewFilter` と `horses` はPlanへ移す。

8.4 破損・未知バージョン

- JSON parse失敗: 元値を保持し、空データで上書きしない。
- `schemaVersion > 4`: 未対応の将来形式として書き込みを禁止する。
- 必須ID欠落・重複: 書き込み前検証を失敗させる。
- コレクション型不正: 破損として扱い、元値を保持する。

8.5 バックアップ

Proposal P6

初回v4書き込み直前に元文字列を `hitokuchiPlanner.card.v5.fixed.backup.v3` へ1回だけ退避する。バックアップ成功後だけ本体を書き込む。容量超過時は移行を中止し、利用者へ通知する。

9. リスクと対応

ID	区分	内容	対応
R1	Fact	現行保存は他募集回を全体置換で失い得る	対象募集回・Planのupsertへ変更
R2	Fact	v3では回共通設定がPlanごとに異なり得る	決定的優先順位と競合診断を使用
R3	Assumption	Plan IDを募集回スコープとする	バリデーションとテストで保証
R4	Proposal	単一localStorageキーを維持する	容量が上限へ近づいたら再評価
R5	Fact	現行は読込・保存例外を黙殺する	移行Issueでエラー表示と書き込み停止を追加
R6	Fact	カタログと選択を分離する決定がある	HTMLカタログ情報をJSONへ複製しない

10. 後続Issue分割

Issue A: schemaVersion 4と移行

- v4モデル、検証、migrateToV4()、バックアップを実装
- v3の設定競合診断を実装
- 完了条件: 移行が冪等で全募集回・Plan・馬選択を保持する

Issue B: 募集回カタログと選択UI

- 安定したround IDと募集回セクターを追加
- Rootの activeRecruitmentRoundId と同期
- 完了条件: 複数回を選択でき、回設定とPlan集合が切り替わる

Issue C: 募集回配下のPlan管理

- Plan追加・改名・削除を選択募集回内へ限定
- 各Recruitment Roundの activePlanId と同期
- 完了条件: Plan操作が他募集回へ影響しない

Issue D: 保存・適用責任の分離

- 募集回設定capture/applyとPlan状態capture/applyを分離
- getPlanData() の全体置換を廃止し対象エンティティをマージ
- 完了条件: 設定・filter・馬選択が承認済み所有者へだけ保存される

Issue E: 統合・移行テスト

- 3募集回 × 各2 Planの切替、再読込、CRUDを検証
- v3競合、破損JSON、未知版、容量超過を検証
- 完了条件: データ混線・欠落がなく旧データを復元できる

依存順序は A -> B -> C -> D -> E とする。後続Issue数は 5件 である。

11. Acceptance Criteria

- [x] 現行永続化モデル、getPlanData()、applyPlan() を調査した。
- [x] Product Ownerの5決定をFactとして反映した。
- [x] 募集回を親、Planを子とする階層へ改訂した。
- [x] 募集回設定とPlan設定の所有者を分離した。

- [x] Rootと募集回のactive ID配置を定義した。
- [x] 3つの設計案と比較を承認済み階層に合わせて更新した。
- [x] 推奨JSON、保存・読込・切替、移行戦略を更新した。
- [x] 後続Issueを5件へ再編した。
- [x] Fact、Assumption、Proposalを区別した。
- [x] コード変更、コミット、ブランチ作成、PRを行っていない。

12. 参照

- Issue #73指示書: E:\Document¥Codex_Issue73_Instruction.pdf
- Architecture Review Revision Instruction v1.1: E:\Document¥Issue73_Architecture_Review_Revision_Instruction_v1.1.pdf
- 現行実装: E:\Document¥Develop¥turflight-planner¥index.html（コミット b739ee2）
- リポジトリ規則: E:\Document¥Develop¥turflight-planner¥AGENTS.md
- 成果物提出標準: Codex Development Standard.md

13. 変更履歴

Version	Date	Status	Changes
1.1.0	2026-07-12	Approved	Product Owner決定に従い募集回を親、Planを子へ変更。設定所有、active ID、代替案、JSON、保存・読込、移行、後続Issueを改訂。Product Owner承認後、正式成果物として登録。
1.0.0	2026-07-12	Superseded	Issue #73の初回調査、3案比較、推奨データモデル、移行戦略、JSON例、後続Issue分割を作成。